

C++ 프로그래밍 과제 4

연산자 오버로딩: C++과 Python 비교 분석

과제 목적

이 과제의 목적은 C++의 연산자 오버로딩(operator overloading) 메커니즘을 이해하고, Python의 대응 기능과 비교함으로써 언어 설계 관점에서의 차이를 파악하는 것이다.

본 과제를 통해 다음을 이해하는 것을 목표로 한다.

- C++에서 연산자를 오버로딩하는 문법과 규칙
- Python의 특수 메서드(dunder method)와의 대응 관계
- 두 언어의 설계 철학 차이
- 연산자 오버로딩이 실제 코드 가독성에 미치는 영향

제출 형식

보고서 (PDF)

보고서는 다음 구조를 포함해야 한다.

- 이론 조사 (각 연산자 그룹별 C++ / Python 비교)
- 구현 코드
- 실행 결과
- 분석 및 결론

코드는 Appendix에 첨부한다.

1. 이론 조사

다음 연산자 그룹 각각에 대해, C++ 문법과 Python 대응 메서드를 정리하고 차이점을 설명하시오.

(1) 산술 연산자

C++에서 +, -, *, /, % 연산자를 오버로딩하는 방법을 설명하시오.

- 멤버 함수 방식과 비멤버 함수 방식의 차이
- Python의 __add__, __sub__, __mul__, __truediv__, __mod__ 와의 대응 관계
- Python의 **reflected 연산자** (__radd__ 등)에 해당하는 C++ 개념이 있는가

(2) 복합 대입 연산자

C++의 +=, -=, *=, /= 를 오버로딩하는 방법을 설명하시오.

- Python의 __iadd__, __isub__ 등과의 대응 관계
- C++에서 operator+= 를 구현하면 operator+ 도 자동으로 생기는가? Python은 어떠한가?
- 반환 타입으로 *this 를 반환하는 이유

(3) 비교 연산자

C++의 ==, !=, <, >, <=, >= 를 오버로딩하는 방법을 설명하시오.

- Python의 __eq__, __ne__, __lt__, __le__, __gt__, __ge__ 와의 대응 관계
- C++에서 operator== 를 정의했을 때 operator!= 가 자동으로 생기는가? (C++20 이전 / 이후 비교)
- Python에서 __eq__ 만 정의하면 != 는 어떻게 동작하는가

(4) 단항 연산자

C++의 단항 -, +, ~ 를 오버로딩하는 방법을 설명하시오.

- Python의 __neg__, __pos__, __invert__ 와의 대응 관계
- 이항 operator- 와 단항 operator- 를 C++에서 어떻게 구별하는가

(5) 첨자 연산자

C++의 operator[] 를 오버로딩하는 방법을 설명하시오.

- const 버전과 비-const 버전을 모두 제공해야 하는 이유
- Python의 __getitem__ , __setitem__ 과의 대응 관계
- Python은 __getitem__ 과 __setitem__ 이 분리되어 있는데, C++에서는 어떻게 읽기/쓰기를 구분하는가

(6) 함수 호출 연산자

C++의 operator() 를 오버로딩하는 방법을 설명하시오.

- 이를 구현한 객체를 함수 객체(functor)라고 부르는 이유
- Python의 __call__ 과의 대응 관계
- 람다(lambda)와 함수 객체의 관계

(7) 출력 연산자와 문자열 변환

C++의 operator<< 를 std::ostream 에 대해 오버로딩하는 방법을 설명하시오.

- 왜 멤버 함수가 아닌 비멤버 함수로 정의해야 하는가
- std::ostream&을 반환하는 이유 (체이닝: cout << a << b)
- Python의 __str__ , __repr__ 과의 공통점과 차이점을 정리하시오

관점	C++ operator<<	Python __str__
출력 대상	ostream 인자로 결정	호출하는 쪽이 결정
체이닝	cout << a << b 가능	해당 없음
다양한 스트림	cout , cerr , fstream 모두 동작	str() 결과를 원하는 곳에 전달

(8) bool 변환 연산자

C++의 operator bool() 을 오버로딩하는 방법을 설명하시오.

- explicit 키워드를 붙여야 하는 이유
- Python의 __bool__ 과의 대응 관계
- if (obj) 구문이 두 언어에서 각각 어떻게 동작하는가

(9) C++에만 있는 연산자 (Python 대응 없음)

다음 연산자들이 Python에서 지원되지 않는 이유를 언어 설계 관점에서 설명하시오.

C++ 연산자	Python에 없는 이유
++, --	
->	
&&, \ \	
new , delete	

2. 프로그래밍: Fraction 클래스 구현

유리수(분수)를 표현하는 Fraction 클래스를 C++과 Python으로 각각 구현하시오.

2-1. C++ 구현

다음 인터페이스를 갖는 Fraction 클래스를 구현하시오.

```
class Fraction {
public:
    Fraction(int numerator, int denominator);

    Fraction operator+(const Fraction& rhs) const;
    Fraction operator-(const Fraction& rhs) const;
```

```

Fraction operator*(const Fraction& rhs) const;
Fraction operator/(const Fraction& rhs) const;

Fraction operator-() const;           // 단항 부호 반전

Fraction& operator+=(const Fraction& rhs);

bool operator==(const Fraction& rhs) const;
bool operator!=(const Fraction& rhs) const;
bool operator< (const Fraction& rhs) const;

explicit operator bool() const;       // 분자가 0이면 false

int operator[](int index) const;      // 0: 분자, 1: 분모

friend std::ostream& operator<<(std::ostream& os, const Fraction& f);

private:
    int num_;
    int den_;
    void reduce();                     // 기약분수로 약분
};

```

구현 조건:

- 생성자에서 분모가 0이면 예외를 던지거나 assert 로 처리한다
- 모든 Fraction 객체는 항상 기약분수 형태로 유지한다 (reduce() 활용)
- 분모는 항상 양수로 유지한다 (Fraction(1, -2) 는 내부적으로 (-1, 2) 로 저장)
- operator<< 는 3/4, -1/2, 2 (분모가 1이면 정수처럼) 형식으로 출력한다
- 1/2 == 2/4 가 true 가 되어야 한다

main 예시:

```

int main() {
    Fraction a(1, 2);
    Fraction b(1, 3);

    std::cout << a << " + " << b << " = " << (a + b) << std::endl;
    std::cout << a << " * " << b << " = " << (a * b) << std::endl;
    std::cout << a << " == " << Fraction(2, 4) << " : "
               << std::boolalpha << (a == Fraction(2, 4)) << std::endl;

    if (a) std::cout << a << " is nonzero" << std::endl;

    a += b;
    std::cout << "a += b : " << a << std::endl;

    std::cout << "분자: " << a[0] << ", 분모: " << a[1] << std::endl;
}

```

예상 출력:

```

1/2 + 1/3 = 5/6
1/2 * 1/3 = 1/6
1/2 == 1/2 : true
1/2 is nonzero
a += b : 5/6
분자: 5, 분모: 6

```

2-2. Python 구현

동일한 Fraction 클래스를 Python으로 구현하시오.

(단, fractions.Fraction 표준 라이브러리는 사용하지 않는다.)

```

class Fraction:
    def __init__(self, numerator: int, denominator: int):
        ...

    def __add__(self, other): ...
    def __sub__(self, other): ...
    def __mul__(self, other): ...
    def __truediv__(self, other): ...

    def __neg__(self): ...

    def __iadd__(self, other): ...

    def __eq__(self, other): ...
    def __lt__(self, other): ...

    def __bool__(self): ...

    def __getitem__(self, index): ... # 0: 분자, 1: 분모

    def __str__(self): ...
    def __repr__(self): ...

```

main 예시:

```

a = Fraction(1, 2)
b = Fraction(1, 3)

print(f"{a} + {b} = {a + b}")
print(f"{a} * {b} = {a * b}")
print(f"{a} == {Fraction(2, 4)} : {a == Fraction(2, 4)}")

if a:
    print(f"{a} is nonzero")

a += b
print(f"a += b : {a}")
print(f"분자: {a[0]}, 분모: {a[1]}")

```

3. 분석 및 결론

보고서 마지막에 다음 질문에 답하시오.

1. C++의 `operator<<` 와 Python의 `__str__` 은 "출력 형식을 정의한다"는 공통점이 있다. 두 접근 방식의 **설계 철학의 차이**를 설명하시오.
2. C++의 `operator[]` 는 하나의 함수로 읽기와 쓰기를 모두 처리하지만, Python은 `__getitem__` 과 `__setitem__` 으로 분리되어 있다. C++에서는 어떤 메커니즘으로 이를 구분하는지 설명하시오.
3. `Fraction` 클래스를 두 언어로 구현하면서 느낀 **가장 큰 차이점**을 자유롭게 서술하시오.